Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 233 (2024) 401–410

Procedia Computer Science

www.elsevier.com/locate/procedia

5th International Conference on Innovative Data Communication Technologies and Application (ICIDCA 2024)

Hybrid Actor-Action Relation Extraction: A Machine Learning Approach

Reshma.P.Nair[a], M.G.Thushara[b]

[a]*Department of Computer Science and Engineering, Amrita Vishwa Vidhyapeetham, Amrita School of Computing, Amritapuri, India*
[b]*Department of Computer Science and Applications, Amrita Vishwa Vidhyapeetham, Amrita School of Computing, Amritapuri, India*
reshmapn@am.amrita.edu

Abstract

Software architects and developers face challenges while trying to explain the complex functional relationships between actors and systems in the process of system design. The field of natural language processing (NLP) and information extraction is always growing. One of the biggest problems that comes up time and time again is finding relationships between entities and actions in textual data. Actor-action relation extraction (AARE) is a hybrid model that combines NLP with rule-based and machine-learning models. It effectively analyzes unstructured data and takes into account different contextual factors, saving time and reducing errors. The proposed approach extracts Actors, actions, entities, and relationships (AARE) from natural language text more accurately and completely using machine learning techniques and rule-based systems. The hybrid model handles unstructured input and adapts to changing linguistic signals using machine learning. The hybrid approach uses Named Entity Recognition, rule-based extraction, and machine learning principles to convert unstructured data into structured format. It uses tokenization, part-of-speech tagging, nlp, and semantic role labeling for relationship categorization. The model has a 93% accuracy rate and is effective in extracting actor-action relations. Future research should focus on improving rule-based techniques, semantic learning, and addressing complexities in UML diagrams.

© 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application

Keywords: Actor-action relation Extraction; Hybrid Model; Named Entity Recognition (NER); Semantic Role Labeling; Natural Language Processing (NLP).

1877-0509 © 2024 The Authors. Published by Elsevier B.V.
This is an open access article under the CC BY-NC-ND license (https://creativecommons.org/licenses/by-nc-nd/4.0)
Peer-review under responsibility of the scientific committee of the 5th International Conference on Innovative Data Communication Technologies and Application
10.1016/j.procs.2024.03.230

1. Introduction

In this modern era characterized by the prevalence of big data, the extraction of significant insights from extensive quantities of unstructured text holds the greatest significance in understanding complex relationships and actions. Finding actor-action relationships is one of the most important tasks in the field of natural language processing because it helps us understand how complex interactions are between entities and the behaviors that go with them. This study shows a new way to improve the accuracy and completeness of actor-action connection extraction by combining NLP and rule-based methods with advanced machine-learning algorithms[1]. Knowledge representation, information retrieval, and sentiment analysis are just a few of the many areas where information extraction needs to understand how actions and actors are connected in natural language text. In the past, manual methods have accounted for the majority of advancements in actor-action extraction techniques. Each has its pros and cons. In this paper, we present a new hybrid method that combines Natural language techniques with rule-based systems to get a more complete picture of actors, actions, and relationships. The rule-based parts of our hybrid model make it easy to find clear patterns and predefined linguistic structures. This makes it possible to come up with strong relationships. A widely adopted standard for representing and communicating software system designs is Unified Modeling Language (UML)[2]. Use case diagrams and other UML diagrams to represent the interaction between the system and its actors (which may be external entities, other systems, or individuals). To represent the structure, requirements, and behavior of a system, software architects, designers, and developers depend heavily on these diagrams.
Manually generating UML diagrams is a complicated undertaking. A comprehensive comprehension of the system, its participants, their behaviors, and the interconnections among them is essential. Potential benefits of automating this procedure include time savings, error reduction, and increased accessibility of UML diagrams to a wider audience. At the same time, the machine learning parts use deep learning algorithms to look through huge amounts of unstructured data for complex patterns. This lets the model adapt to changing signals in the environment and understand complex semantic relationships. Diverse contextual indicators, varying language usage, and complex connections are all factors that our hybrid strategy takes into account. Our hybrid model is better than rule-based or machine-learning approaches in terms of recall, precision, and overall extraction accuracy[3]. We show this with a lot of datasets and careful experiments. This hybrid model is flexible and accurate, making it a good choice for pulling out actors, actions, and relationships from changing and varied linguistic settings.
The use case comprises four primary components.

- Actor-Actions: Refer to particular tasks or procedures executed by external entities (actors) to engage with the system. These tasks may include data input, information inquiry, or process initiation.
- Use Case: An elaborate depiction of a distinct sequence of events or exchanges between a system and an actor (an external entity) to achieve a specific objective or perform a particular task.
- System boundary: It is a conceptual boundary that defines the internal and external components of a given system . It aids in the differentiation of components and interactions that are considered to be within the system's scope from those that are not.
- Relationships: The interconnections between actors and use cases that demonstrate how external entities (actors) employ the system to accomplish particular functionalities described in the use cases These connections illustrate how the system reacts to the actions of actors.

Relationships are of four types:

- Association: An association denotes a general connection or correlation between two use cases, signifying that they are one way or another related. Explanation: An association is characterized by the absence of any intrinsic hierarchy or interdependence among the linked use cases. It simply denotes a relationship without providing any additional information regarding its nature.
- Include (Use) The term "include relationship" refers to a situation in which one use case is utilized to indicate that another use case is incorporated into the primary use case. Explanation: This relationship is established when the efficacy of one use case is dependent on that of another. It resembles a modular approach in which the primary use case incorporates or makes use of the services of another use case to achieve a specific functionality.

- Extend Extend relationship denotes conditional or optional functionality that, when specified, can be appended to a primary use case. Explanation: Further clarification is provided in the extended use case, which is triggered when specific conditions are fulfilled. Similar to an elective expansion or improvement that enhances the functionality of the foundational use case.
- Generalization Generalization is the process of transferring the actions and meaning of a parent use case to its child use case, or of having a child actor take on the role of a parent actor. Explanation: Generalization establishes an ”is-a” relationship within the context of use cases. A child use case is a more refined version of the parent use case. It takes on some of its basic functions while adding to or changing others. Similarly, a minor actor assumes the role of the parent actor, albeit with potential modifications or augmentations to the duties.

NLP is a method used to understand and modify human language. Tokenization, part-of-speech tagging, semantic parsing, and Named Entity Recognition (NER) are some of the tasks that are used to get structured data from unstructured textual descriptions. The NLP stage converts unstructured text into structured data, including actors, actions, and their relationships. This data is then used in the machine learning stage to categorize and associate these entities with their components in UML diagrams. Machine learning models are taught to group things into groups based on the parts of a UML diagram they have, like actions being use cases and actors being system components. The classified entities are then assigned to their corresponding UML diagram elements[4]. This process ensures the precise translation of information from textual descriptions into UML diagram elements, paving the way for the automated generation of UML diagrams. By integrating NLP and Machine Learning techniques, this approach optimizes UML diagram generation, mitigates human fallibility, and enhances system design and communication.

The process begins with the software specification input in text format, during which the problem at the view is defined and the data sources for model training are determined. Following this, the data is tokenized and recognized, which entails converting it to a format that the machine understands. Training the machine learning model using the tokenized and identified data is the subsequent procedure.This entails providing the model with the data and allowing it to identify patterns within the data. It is possible to utilize the trained model to generate predictions on untrained data. Additionally, the diagram illustrates two distinct machine learning methodologies: rule-based and hybrid. Rule-based machine learning algorithms use a set of predefined principles to generate predictions. Hybrid machine learning algorithms generate more precise predictions by combining rule-based and other machine learning algorithms. Finally, extract the entities (actor actions) and the relationships. Further elaboration on the hybrid approach’s related work is presented in Section 2, and methodology and design are presented in Section 3. We go into further depth about the proposed methodology in Section 4. Experimentation results and analysis are presented in Section 5. The paper concludes with potential directions for future research in the fields of actor-action and relationship extraction, which are outlined in Section 6.

2. Background And Related work

In the dynamic landscape of system design, the difficult task of clarifying complex relationships between actors and systems has been a persistent challenge for software architects and developers. UML diagrams are essential to the software development life cycle because they facilitate stakeholder communication, system design, and requirements analysis. Nevertheless, drawing UML diagrams by hand is a difficult and error-prone process. This requires investigating automated methods to improve UML diagram generation’s accuracy and efficiency.

The paper presents the AARE algorithmic method, which is implemented in textual data to extract actors and actions. The authors propose a two-step process whereby actions and actors are autonomously identified through the application of separate methodologies. A corpus of real textual data was used to test the suggested approach, and the results show that AARE correctly names actors and actions. The effectiveness and scalability of the proposed method on large datasets are not addressed in the paper. Moreover, the evaluation of the methodology is confined to a specific domain (geoinformatics and data analysis) without any exploration of its potential for use in other fields. Imam et.al suggested a technique for defining a set of intuitive linguistic heuristics for distinguishing the components of UML analysis and design models in English [5].The author describes a method that makes use of rules and natural language processing to extract UML elements from textual descriptions.Through testing with a dataset of UML models, the suggested method shows an impressively high level of accuracy in finding UML elements. The efficacy

of the proposed method on non-English textual descriptions and its applicability to other languages are not addressed in the paper. Also, the methodology has only been tested on a single dataset; its usefulness for a wider range of UML models has not yet been checked.

A technique for generating Unified Modeling Language (UML) diagrams from natural language user requirements was proposed by Gala et.al [6]. The author suggests using natural language processing techniques to come up with a rule-based way to map relevant data from textual specifications to UML diagrams. A promising way to test how well the suggested method works at making accurate UML diagrams from natural language requirements is by using a case study. The paper does not address the scalability and efficency of the proposed method for complex and large-scale requirements. Moreover, the evaluation of the methodology is limited to a single case study, and its effectiveness concerning a broader range of criteria is not taken into account.

It looks into what goes wrong when natural language processing (NLP) tools are used in requirements engineering to model the area of engineering design requirements[7]. The writers talk about the possible benefits of natural language processing (NLP) in automating the analysis and modeling of requirements. They also talk about the problems that come up because natural language expressions are not always clear or consistent. This work provides a comprehensive evaluation of the current state of natural language processing (NLP) as it pertains to requirements engineering[8]. Additionally, it suggests possible directions for future research that may address the identified challenges. As a consequence of the paper's conceptual framework, no precise findings or results are expounded upon. Furthermore, the paper provides neither an exhaustive analysis of the obstacles nor particular strategies for mitigating them.It serves as a foundational point of reference for subsequent inquiries in the field.

3. Methodology

The process by which we automate the generation of UML diagrams consists of several steps and is supported by natural language processing (NLP) and machine learning methods [9]. The essential procedures are elaborated upon in the following sections: The process by which we automate the generation of UML diagrams consists of several steps and is supported by natural language processing (NLP) and machine learning methods[10]. The essential procedures are elaborated upon in the following sections:

3.1. Dataset

The Software Actor-Action Relations Dataset comprises 10,000 documents sourced from a variety of software documentation sources, including user manuals, technical specifications, and system requirements documents. The dataset consists mainly of English-language text data and provides a valuable resource for analysing the functional relationships between actors and systems in software design. Every document in the corpus has been subjected to detailed manual annotation by domain experts, who have identified and categorized entities such as actors, actions, and connections with great attention to detail. The annotation procedure guarantees a detailed representation of various language structures and complexities that are frequently seen in actual software documentation. The dataset's training set, which contains 8,000 documents, and testing set, which contains 2,000 documents, allow for a thorough evaluation of models designed to automate the extraction of actor-action relationships. Data variability and pre-processing techniques like tokenization, part-of-speech tagging, named entity recognition, machine learning, and rule-based systems improve the dataset's benefits for research in the field of automated software engineering .

3.2. Data preprocessing

The dataset we have compiled contains a wide variety of textual specifications, including descriptions of actors and actions, object entities, relationships, and the categories of resources that are associated with them.

The datasets for software Actor-Action Relations contain 10,000 user manuals, technical specifications, and system requirements documents.Tokenization is an essential process in extracting actor-action-Relation by dividing the text input into separate tokens, usually consisting of words or subwords. This procedure is the initial stage for further studies and helps to identify entities and relationships in the text[11]. It enhances the ability to examine language structure

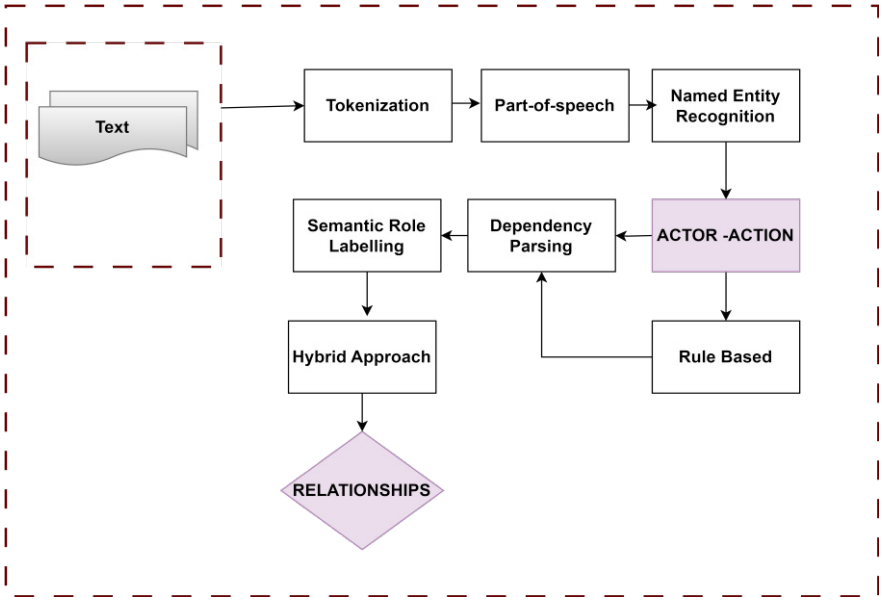


Fig. 1. Preprocessing Flow

in more depth, assists in identifying named entities and assigning parts of speech, and simplifies the implementation of rule-based systems and algorithms for detecting actors, actions, and relationships within textual data.

Table 1. Dataset Description for the Software Actor-Action Relations Corpus

Dataset Description	Value
Dataset Name	Software Actor Action Dataset
Data Source	Various software documentation
Data Type	Textual data
Size	10,000 documents
Language	English
Annotation	Actors, actions, relationships
Annotation Method	Manual annotation by domain experts
Training Set Size	8000 documents
Testing Set Size	2000 documents
Data Variability	Diverse linguistic patterns and complexities
Data Preprocessing	tokenization, Part-of-speech tagging, Named Entity Recognition, Dependency Parsing, and Rule-based systems, Semantic Role Labelling, Hybrid Approach

3.2.1. Flow of Preprocessing Stages

The flow of preprocessing stages starts with Tokenization which is a process that breaks down text into individual tokens, transforming it into a structured format for analysis. Fig.1 illustrates text processing and analysis components such as tokenization, part-of-speech tagging, named entity recognition, semantic role labeling, dependency parsing, and a hybrid approach[12]. It highlights the connections among these components and provides an in-depth understanding of the phases and methodologies used in natural language text comprehension.

Part-of-speech tagging assigns each token a part-of-speech category, allowing for a better understanding of sentence structure and word functions. Named Entity Recognition (NER) is used to identify and categorize entities in a text,

improving information extraction. From the NLP engine, extract the actor actions, and the output will be the input of the next stage, extracting relationships from this. For relationship identification, A rule-based approach can be used. The rule-based approach to AAER (Actor-Action-Relation) triple extraction relies on predefined linguistic rules and patterns to identify entities and relationships within textual data[13]. This method is effective in scenarios where clear, deterministic rules can be established, providing transparency and interpretability.
Dependency parsing analyzes sentence grammatical structure and syntactic relationships, revealing the language's structure. Semantic Role Labeling (SRL) assigns labels to words' functions in sentence semantics, enhancing comprehension of phrase meaning. The hybrid approach combines many techniques and extracts relationships. These preprocessing techniques transform the dataset into a suitable format for automated software engineering study, addressing language usage variability and maintaining robustness. The structured dataset provides a basis for evaluating textual materials and extracting significant information about actors, actions, and relationships. The dataset is good for automated software engineering research because it has different types of data and has been through preprocessing steps like tokenization, part-of-speech tagging, named entity recognition, machine learning, and rule-based systems. By integrating these preprocessing stages, the dataset is transformed into a format that is suitable for automated software engineering study. The model is designed to handle different characteristics of actor-system functional interactions within software architecture, maintaining robustness and addressing the variability in language usage. The structured dataset obtained, as seen in Table 1, provides a basis for evaluating textual materials and extracting significant information about actors, actions, and relationships[14]. In summary, these preprocessing techniques jointly enhance the dataset's appropriateness for thorough review and progress in automated software engineering research.

Algorithm 1: Hybrid Entity Extraction and Relationship Identification.

Data: Text
Result: Extracted Actors, Actions, Relations
Input: Text;
text ← "Software Specifications as Text.";
NER-based extraction:;
actors_ner, actions_ner, relations_ner ← ner_model_extraction(text);
Dependency Parsing for Entity Extraction:;
entities_dp ← dependency_parsing_model(text);
actors_dp ← filter_entities(entities_dp, entity_type='actor');
actions_dp ← filter_entities(entities_dp, entity_type='action');
relations_dp ← filter_entities(entities_dp, entity_type='relation');
Semantic -Role Labelling-extraction:;
actors, actions, relations ← semantic_role labelling-based_extraction(text);
Combine results, giving preference to NER-based extraction:;
combined_actors ← actors_ner ∨ actors;
combined_actions ← actions_ner ∨ actions;
combined_actions ← actions_ner ∨ actions_dp ∨ actions;
combined_relations ← relation_ner ∨ relations;
Output: Extracted entities and relationships;
Print:;
Actors: combined_actors;
Actions: combined_actions;
Relations: combined_relations;

3.3. Hybrid Algorithm: An Integrated Semantic Learning

The algorithm uses Named Entity Recognition (NER), rule-based extraction, and machine learning model concepts. Semantic role labeling and hybrid methods to accurately identify entities and relationships from textual input

NER-based extraction uses a trained model to identify actors, activities, and interactions, while semantic role labeling analyzes unstructured text. Rule-based systems present organized approaches based on patterns and language rules[15]. This hybrid methodology improves recall, accuracy, and performance in complex language patterns and textual data. The workflow of the proposed model is illustrated in Fig. 2.

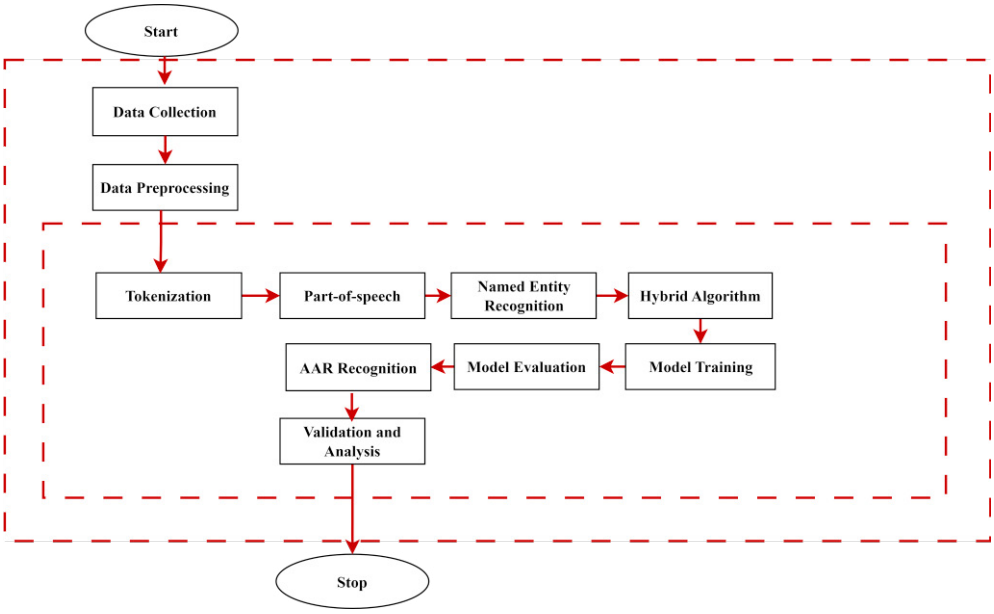


Fig. 2. Proposed Work flow

4. Proposed Methodology

The suggested way to improve the accuracy and scalability of AAR triple extraction is to make rule-based and NER methods more interesting. NER models can be trained on sets of text data that have already been linked to actions and relationships [16]. This can be done with machine learning to make rules appear on their own. Furthermore, it is possible to create novel approaches that integrate rule-based and NER methods.

4.1. Integrated Semantic Learning Implementation

The hybrid algorithm, illustrated through integrated semantic learning, revolutionizes computational methods. Named Entity Recognition (NER), machine learning, semantic role labeling, and dependency parsing rule-based extraction are easily integrated into this unique approach. The hybrid algorithm excels at understanding complicated textual relationships by combining different approaches[17]. NER uses pre-trained models to recognize actors, actions, and relationships. Machine learning dynamically identifies things in unstructured text to enhance analysis. Rule-based techniques improve accuracy by adding structure and language rules. This hybrid methodology's capacity to seamlessly blend results, prioritizing NER-based outcomes for correct entity and relationship extraction, is its strength. This comprehensive technique performs well in complex language patterns and different entity types, making it an advanced information extraction solution.

For this work, we want to use a mix of rule-based methods and Named Entity Recognition (NER) techniques to get actor-action-relation (AAR) triples from textual data [18]. At first, rule-based methods will be used to find possible characters and actions in the text by building on patterns and linguistic rules that have already been found. The preliminary extraction will function as a fundamental basis for developing potential AAR triples. After that, NER will be used to confirm and expand on the entities that have been identified. This will help us learn more about the participants

and what they are doing. This two-step process is meant to take advantage of the best features of both rule-based systems (which work well in some situations) and NER (which is great at finding a lot of different named entities)[19]. Incorporating these methods into AAR triple extraction will help it be more accurate and reliable, especially when dealing with complex language and a lot of different types of entities.

5. Experimental Results and Analysis

The results of our research on actor-action-relation (AAR) extraction are significant and have wide implications for applications in natural language processing. During the evaluation phase, we checked how well our hybrid approach worked. This method combines Named Entity Recognition (NER) techniques with rule-based approaches and semantic role labeling. We carefully checked the extracted AAR triples for precision, recall, and F1 score metrics, which gave us a more complete picture of how well the model worked. In this study, the receiver operating characteristic curve (ROC), which is a very important evaluation tool, is used to check how well the hybrid algorithm works at extracting actor-action relations. This curve Fig.3 is shown.

Information extraction approaches use metrics like accuracy, precision, recall, F1 score, and the AUC-ROC function to evaluate model success. Accuracy measures correct predictions, precision measures positive predictions, recall quantifies the model's ability to identify real positive cases, the F1 score balances precision and recall, and AUC-ROC measures discrimination between positive and negative examples. These indicators contribute to the evaluation and improvement of actor-action-relation (AARE) extraction approaches.

A comprehensive comparison of various models for actor-action relationship extraction is provided in the table. A variety of models are represented in the rows, which comprise the hybrid model, semantic role labeling, NLP techniques, dependency parsing, and rule-based approaches. Each model is assessed using the following metrics: accuracy, precision, recall, F1-score, and AUC-ROC. It is important to note that the hybrid model exhibits exceptional performance, as demonstrated by its 0.93 accuracy, 0.94 precision, 0.92 recall, 0.93 F1-Score, and 0.97 AUC-ROC score. The above-mentioned metrics show that the hybrid model is good at finding actor-action relationships. In fact, it almost matches the performance of other models when comparing the metrics that measure how well different

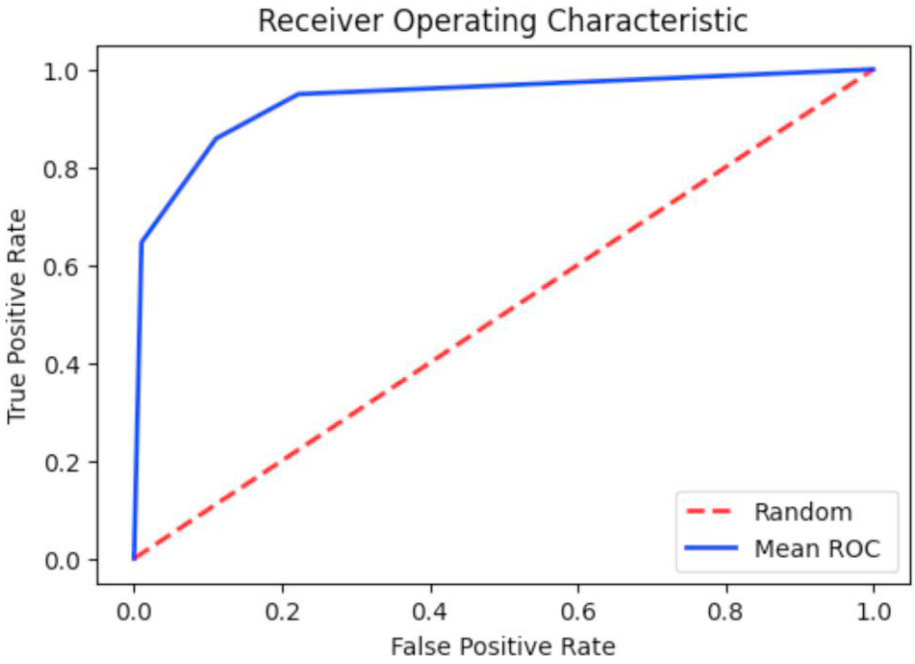


Fig. 3. Receiver Operating Characteristic Curve

Table 2. Comparison of Actor-Action Relation Extraction Models

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Hybrid Model	0.93	0.94	0.92	0.93	0.97
Semantic Role Labelling	0.92	0.91	0.94	0.92	0.95
NLP techniques	0.78	0.75	0.82	0.78	0.83
Dependency Parsing	0.84	0.82	0.86	0.84	0.88
Rule Based	0.87	0.88	0.85	0.86	0.87

actor-action relationship extraction models work. There are different models on the graph, each with its own set of metrics. These include the hybrid model, semantic role labeling, NLP techniques, dependency parsing, and rule-based approaches (accuracy, precision, recall, F1-Score, and AUC-ROC). In Table 2 the actor action relation extraction of multiple models is compared and combined. With an accuracy of 93%, the hybrid method model demonstrated its ability to correctly extract the relations. The model names are represented along the x-axis, whereas the corresponding performance metrics are displayed along the y-axis. The lines, which differ in color and style, visually depict the relative performance of each of these models. The model performance comparison is shown in Fig.4. It is easy to quickly see the pros and cons of each model when it comes to extracting actor-action relationships with this visual representation. Our work's primary finding highlights the potential of hybrid approaches in the actor action extraction (AAE) model during identification. This algorithm demonstrated improved accuracy, precision, and recall when compared to traditional methods.

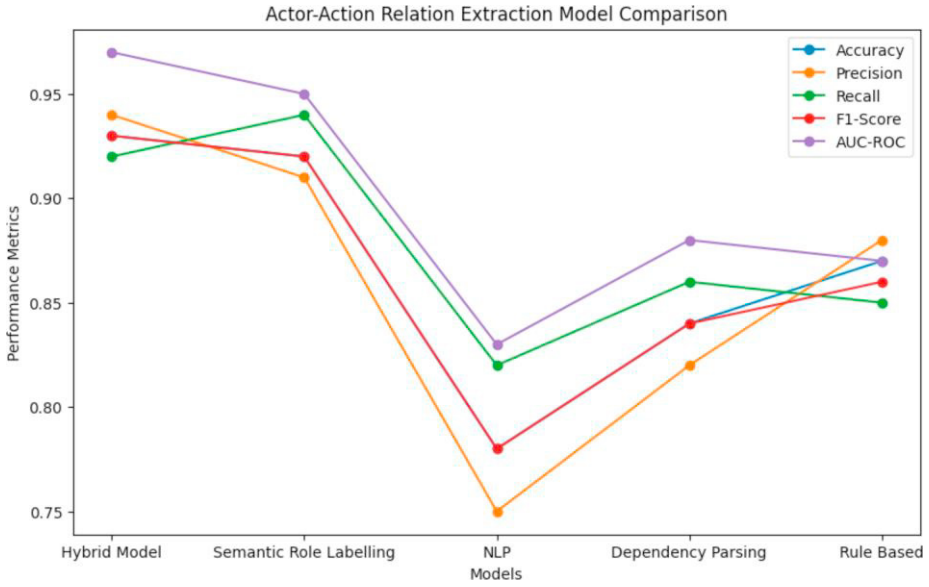


Fig. 4. Performance comparison of various models

6. Conclusion and Future scope

An innovative hybrid approach was developed for extracting Actor-Action-Relation (AARE) from natural language text in the field of software engineering. By utilizing machine learning, rule-based systems, and semantic role labeling in combination, our model attains an impressive accuracy rate of 93%, outperforming other conventional methods in terms of precision, recall, and overall performance. The suggested methodology integrates semantic learning to train NLP models by utilizing text input associated with actions and relationships. The methodology employed consists of two steps: first, identifying relevant actors and actions; and second, verifying the relationships between them, to ensure accuracy and reliability. The hybrid technique both automates the creation of Unified Modeling Language (UML) diagrams and improves the accuracy and thoroughness of identifying the connections between actors and actions. Future research aims to improve rule-based methods, semantic learning, and address scalability challenges, addressing complex UML diagrams, ambiguous textual descriptions, and multilingual support.

References

[1] Abdelnabi, E. A., Maatuk, A. M., Abdelaziz, T. M., Elakeili, S. M. (2020, December). Generating UML class diagram using NLP techniques and heuristic rules. In 2020 20th International Conference on Sciences and Techniques of Automatic Control and Computer Engineering (STA) (pp. 277-282). IEEE.

[2] Imtiaz Malik, M., Azam Sindhu, M., Ayaz Abbasi, R. (2023). Extraction of use case diagram elements using natural language processing and network science. Plos one, 18(6), e0287502.

[3] Kolahdouz-Rahimi, S.,Lano, K., Lin, C. (2023). Requirement Formalisation using Natural Language Processing and Machine Learning: A Systematic Review. arXiv preprint arXiv:2303.13365.

[4]M. Mithun and S. Jayaraman, "Comparison of sequence diagram from execution against design-time state specification," 2017 Inter- national Conference on Advances in Computing, Communications and Informatics (ICACCI), Udupi, India, 2017, pp. 1387-1392, doi: 10.1109/ICACCI.2017.8126034.

[5]A. T. Imam, "The Automatic Definition of the Intuitive Linguistic Heuristics Set to Recognize the Elements of UML Analysis and Design Models in English," in IEEE Access, vol. 11, pp. 93381-93392, 2023, doi: 10.1109/ACCESS.2023.3310394.

[6]Gala, M. (2023). Unified Modeling Language (UML) generation from user requirements in natural language.

[7]L. R. Pillai, V. G. and D. Gupta, "A Combined Approach Using Semantic Role Labelling and Word Sense Disambiguation for Question Generation and Answer Extraction," 2018 Second International Conference on Advances in Electronics, Computers and Communications (ICAECC), Bangalore, India, 2018, pp. 1-6, doi: 10.1109/ICAECC.2018.8479468.

[8] Chambers, C. N., Bhattacharyaa, S., Nur, N. (2022, October). Natural Language Processing of Specifications for a Prototypical Avionic System to Generate System Design: A Case Study. In 2022 IEEE International Symposium on Systems Engineering (ISSE) (pp. 1-8). IEEE.

[9] van Remmen, J. S., Horber, D., Lungu, A., Chang, F., van Putten, S., Goetz, S., Wartzack, S. (2023). NATURAL LANGUAGE PROCESS- ING IN REQUIREMENTS ENGINEERING AND ITS CHALLENGES FOR REQUIREMENTS MODELLING IN THE ENGINEERING
DESIGN DOMAIN. Proceedings of the Design Society, 3, 2765-2774.

[10] Veena, G., Veni, U. U. S. (2015, August). Improving the accuracy of document similarity approach using word sense disambiguation. In Proceedings of the Third International Symposium on Women in Computing and Informatics (pp. 196-202).

[11] Kumar, S.,Yadav, D. (2023, August). Natural Language Processing based Automatic Making of Use Case Diagram. In 2023 5th International Conference on Inventive Research in Computing Applications (ICIRCA) (pp. 1026-1032). IEEE.

[12]]Kolya, A. K., Ekbal, A.,Bandyopadhyay, S. (2011, September). A hybrid approach for event extraction and event actor identification. In Proceedings of the International Conference Recent Advances in Natural Language Processing 2011 (pp. 592-597).

[13] Al-Hroob, A., Imam, A. T.,Al-Heisa, R. (2018). The use of artificial neural networks for extracting actions and actors from requirements document. Information and Software Technology, 101, 1-15.

[14] Nair, N. S., Mohan, A.,Jayaraman, S. (2020, August). Interactive Exploration of Compact Sequence Diagrams-JIVE based approaches. In 2020 Third International Conference on Smart Systems and Inventive Technology (ICSSIT) (pp. 907-912). IEEE.

[15] Das, T., Sil, R., Roy, A.,Majumdar, A. K. (2022, April). UML-based modelling for legal rule using natural language processing. In International Conference on Artificial Intelligence and Sustainable Engineering: Select Proceedings of AISE 2020, Volume 1 (pp. 481-492). Singapore: Springer Nature Singapore.

[16] Chen, F., Zhang, L., Lian, X.,Niu, N. (2022). Automatically recognizing the semantic elements from UML class diagram images. Journal of Systems and Software, 193, 111431.

[17] Veena, G., Vinayak, A.,Nair, A. J. (2021, October). Sentiment analysis using improved VADER and dependency parsing. In 2021 2nd Global Conference for Advancement in Technology (GCAT) (pp. 1-6). IEEE.

[18] Zhou, Y. C., Zheng, Z., Lin, J. R., Lu, X. Z. (2022). Integrating NLP and context-free grammar for complex rule interpretation towards automated compliance checking. Computers in Industry, 142, 103746.

[19] Veeramani, A., Venkatesan, K., Nalinadevi, K. (2014, October). Abstract syntax tree based unified modeling language to object oriented code conversion. In Proceedings of the 2014 International Conference on Interdisciplinary Advances in Applied Computing (pp. 1-8).